EF Core: SelectMany()

Concise lesson summary — flattening related collections, comparing SelectMany() with Select() and Join(), with key terms highlighted.

1. Core idea: Select() vs SelectMany()

Select() returns one result per source item; that result may itself be a collection. **SelectMany()** expands child collections into one **flat sequence**.

Mental model: Student A (2 enrollments) + Student B (3) + Student C (1) → SelectMany() → 6 enrollment rows.

```
// Nested collection results
var nested = context.Students
    .Select(s => s.Enrollments);

// Flattened enrollment sequence
var enrollments = context.Students
    .SelectMany(s => s.Enrollments);
```

2. SelectMany() with a result selector

```
var query = context.Students
    .SelectMany(
        student => student.Enrollments,
        (student, enrollment) => new
        {
            student.StudentId,
            StudentName = student.FirstName + " " + student.LastName,
            enrollment.CourseId,
            enrollment.Status
        })
    .OrderBy(x => x.StudentId);
```

• First lambda chooses the child collection to expand: student => student.Enrollments.

• The result selector (student, enrollment) => new { ... } shapes each flat row and can select values from both parent and child.

3. SelectMany() vs Join()

Join() matches two sources using key selectors (for example, StudentId = StudentId). **SelectMany()** is useful when a navigation property already represents a parent-child collection that you want to flatten.

Memory anchor: Join() = match by keys. SelectMany() = expand child collections.

4. SQL translation

```
SELECT s.StudentId,
       s.FirstName + N' ' + s.LastName AS StudentName,
       e.CourseId, e.Status
FROM Students AS s
INNER JOIN Enrollments AS e
    ON s.StudentId = e.StudentId
ORDER BY s.StudentId;
```

• SelectMany() over a navigation collection often translates to a SQL JOIN. The exact SQL depends on the query and model.

• **ToQueryString()** previews the SQL query shape; **ToList()** executes the query and materializes the results.

5. Common mistakes

Mistake: using Select(s => s.Enrollments) when you need individual enrollment rows. This can leave you with a sequence of collections.

Better: use SelectMany(s => s.Enrollments), then project only the fields your report needs.

6. Filtering and practical uses

```
var activeEnrollments = context.Students
    .Where(s => s.Status == "Active")
    .SelectMany(s => s.Enrollments);
```

• Use SelectMany() for enrollment reports, exports, flattening nested data, and traversing child collections across many parents.

Final takeaway

Select() keeps each selector result as one item (which may be a collection). SelectMany() flattens child collections. Join() matches two sources using keys.

Note: the SQL shown follows the lesson example; actual translation can vary with EF Core model, provider, and query shape.